Decomposicao geometrica de um semiprimo - Versao consolidada v3

August 9, 2026

Contents

1 Decomposição geométrica de um semiprimo N = L × q e sua relação com compressão de dados **2**

1.1 1. Motivação . . . 2
1.2 2. Construção geométrica . . . 2
1.3 3. Decomposição em T1 / T2 . . . 2
1.4 4. Leitura em termos de compressão . . . 3
1.5 5. Implementação . . . 3
1.6 6. Resultados . . . 3
1.7 7. Duas técnicas de truncamento sem divisão . . . 4
1.8 8. Fusão de blocos como grafo (union-find) . . . 4
1.9 9. A constante da Tabela 1 não é universal: sensibilidade ao alinhamento binário . . . 4
1.9.1 9.1 Correção: a causa não é a heurística de amostragem . . . 5
1.10 10. Hexágono: dois trapézios colados pela hipotenusa . . . 6
1.11 11. Efeito telescópico entre valores de k (gnomons generalizados) . . . 6
1.12 12. Comparação com dados de ocupação esparsa (esfera e terreno sintéticos) . . . 7
1.12.1 12.1 Por que dados sintéticos . . . 7
1.12.2 12.2 Extensão do método para 3D . . . 7
1.12.3 12.3 Resultados . . . 7
1.12.4 12.4 Interpretação . . . 7
1.12.5 12.5 O piso teórico para dados incompressíveis: um terceiro tipo de truncagem . . . 8
1.13 13. Custo amortizado: construção única vs. renderização por quadro . . . 8
1.13.1 13.1 Por que contagem de nós não é a métrica certa . . . 8
1.13.2 13.2 Metodologia . . . 8
1.13.3 13.3 Ponto de equilíbrio (break-even) . . . 9
1.13.4 13.4 Interpretação . . . 9
1.14 14. O triângulo como figura de referência (Altura Máxima e Projeção Quadrada) . . . 9
1.14.1 14.1 A projeção como quadrado . . . 9
1.14.2 14.2 O quadrado como referência de medida . . . 10
1.14.3 14.3 Consequência prática . . . 10

1.14.414.4 Nota conceitual: medir com referência não muda a figura . . 10
1.1515. Relações Métricas: Leitura Associativa vs. Fórmula 10
1.15.115.1 O teorema 10
1.15.215.2 O que essas relações realmente são: associação antes de fórmula 10
1.15.315.3 Por que essa distinção importa 11
1.1616. Recuperação Geométrico-Métrica: Método da Partição e Escala . . . 11
1.16.116.1 Premissas 11
1.16.216.2 Princípio da escala base 11
1.16.316.3 Recuperação por partição e truncamento 11
1.16.416.4 Síntese do algoritmo 11
1.16.516.5 Vantagem e generalização 12
1.1717. Próximos passos 12
1.1818. Status da implementação em engine (C++) 13

1 Decomposição geométrica de um semiprimo N = L × q e sua relação com compressão de dados

Versão consolidada — funde o relatório principal (seções 1–14) com o adendo sobre o triângulo como figura de referência (seções 15–17), cuja numeração original colidia com a expansão do relatório principal.

1.1 1. Motivação

Um semiprimo $N = L \times q$ pode ser representado por uma malha triangular de pontos: para L fixo, os pontos (x, y) com $0 \le y < L$ e $0 \le x < L - y$ formam um triângulo retângulo de $L(L+1)/2$ pontos. Dentro dessa malha, um bloco quadrado de lado

$$k = L - q$$

concentra a informação sobre o fator q: conhecer k (e L) é equivalente a conhecer q. O objetivo deste relatório é registrar a construção geométrica feita, a decomposição observada em duas regiões em forma de T, e a analogia com técnicas de compressão/renderização que motivou a discussão seguinte.

1.2 2. Construção geométrica

A malha é desenhada em duas projeções: uma vista 2D direta e uma vista 3D em que o bloco central ganha altura k, sugerindo um paralelepípedo "extraído" da malha plana. Exemplo: malha triangular $L = 9$ com bloco central $k = 3$ ($q = L - k = 6$, $N = Lq = 54$).

1.3 3. Decomposição em T1 / T2

Centralizando o bloco de lado k sobre a diagonal de simetria $x = y$ da malha (posição $c = (L - k)/2$), o restante da malha se separa, ponto a ponto, em três classes:

- $T_1 = \{(x, y) : x > y\}$ — bloco próximo ao vértice $(L-1, 0)$;
- $T_2 = \{(x, y) : y > x\}$ — bloco próximo ao vértice $(0, L-1)$, espelho de T_1 pela troca $x \leftrightarrow y$;
- pontos com $x = y$ — a própria dobra de simetria.

Cada um desses blocos tem, visualmente, o formato de um T. Para $L = 9$, $k = 3$: $|T_1| = |T_2| = 18$ pontos e 3 pontos na diagonal, totalizando $45 = \binom{9}{2}$ pontos fora do bloco de 9 pontos (k^2).

1.4 4. Leitura em termos de compressão

A decomposição sugere descrever a malha sem enumerar todos os $L(L+1)/2$ pontos:

1. o bloco central é homogêneo, determinado por três números (L, k, c);
2. T_1 e T_2 são simétricos — basta descrever um e refletir;
3. a malha completa é reconstruída aplicando essas regras, sem processar a região redundante.

Essa é a mesma lógica de voxel culling, sparse octrees e clipmaps: evitar processar regiões homogêneas ou fora de interesse. A diferença é a origem do critério de redutibilidade — aqui algébrico (determinado por L e q); num motor gráfico, geralmente por esparsidade de conteúdo ou visibilidade.

1.5 5. Implementação

Implementado em Python (`compress_demo.py`):

- **Critério de homogeneidade:** região $[x_0, x_0 + s) \times [y_0, y_0 + s)$ é homogênea se os quatro cantos e o centro têm o mesmo valor; para $s \leq 2$ a checagem é exata célula a célula (necessário porque a hipotenusa é uma fronteira fina que pode escapar da amostragem por cantos).
- **Estrutura de dados:** quadtree — cada nó é folha (valor único) ou se subdivide em 4 filhos. Nós homogêneos nunca são expandidos.
- **Regra de reconstrução:** `query(x,y)` desce a árvore até a folha correspondente — nenhum array $L \times L$ é materializado em memória.

Para o caso 3D, o volume é processado fatia por fatia ao longo de z: cada fatia é comprimida em um quadtree independente e descartada antes da próxima (streaming) — pico de memória = tamanho de uma única fatia comprimida.

1.6 6. Resultados

L	k	células (L²)	nós do quadtree	nós/L
16	5	256	89	5.56
64	21	4 096	401	6.27
256	85	65 536	1 673	6.54
1024	341	1 048 576	6 785	6.63

L	k	células (L^2)	nós do quadtree	nós/L
4096	1365	16 777 216	27 257	6.65

Tabela 1: Crescimento do quadtree com L (fixando k ≈ L/3). A razão nós/L estabiliza, confirmando escala O(L).

O experimento de streaming 3D (L=256, k=70) confirma o ganho de memória: volume bruto de 16,8 milhões de células, mas pico de apenas 1.561 nós de uma única fatia comprimida — razão $9{,}3 \times 10^{-5}$. Reconstrução ponto a ponto validada contra a função de ocupação original em 2000 amostras aleatórias por caso, sem divergência. **(Confirmado por execução direta do script `compress_demo-3.py`: pico de 1.561 nós bate exato.)**

1.7 7. Duas técnicas de truncamento sem divisão

Truncamento por máscara (resíduo): $i \,\&\, R_k$ (com $R_k = 2^k - 1$) calcula $i \bmod 2^k$ sem custo de divisão, mas não preserva a ordem de grandeza de i. Usado no protótipo original para evitar divisões/módulos.

Truncamento por deslocamento (shift): $i \gg k$ calcula $\lfloor i/2^k \rfloor$ exatamente, preservando a magnitude. Técnica correta quando o objetivo é dimensionar (decidir se dois valores são "parecidos o bastante").

Teste numérico ($k = 4$, $R_k = 15$): os testes $(i \,\&\, R_k) \leq 1$ e $(i \gg k) = 0$ discordam em 128 dos 256 casos — respondem perguntas diferentes; nenhum é "o errado".

1.8 8. Fusão de blocos como grafo (union-find)

Em vez de subdividir recursivamente, trata-se os blocos como nós de um grafo, fundindo blocos adjacentes cujo valor truncado (por deslocamento) coincide, via union-find — abordagem de Felzenszwalb–Huttenlocher, mais flexível que o quadtree (regiões fundidas não precisam ser quadrados alinhados à grade).

L	blocos totais	componentes	razão
32	64	2	0.0312
128	1 024	2	0.0020
512	16 384	2	0.0001
1024	65 536	2	0.00003

Tabela 2: Fusão de blocos por grafo (kshift=1, blocos 4×4). Componentes O(1), à custa de perder a fronteira exata. **(Confirmado por execução de `merge_grafo.py`.)**

Quadtree preserva a fronteira exata ao custo de O(L); o grafo com truncamento comprime para O(1) classes ao custo de perder a posição exata da fronteira.

1.9 9. A constante da Tabela 1 não é universal: sensibilidade ao alinhamento binário

A hipótese de que nós/L fosse função fechada apenas de k/L não bateu com os dados (discrepância de até ~46%). Variando k/L para $L = 4096$ fixo:

k/L	nós/L
1/4	4.00
1/6	5.32
1/3	6.66
3/8	4.01
1/2	4.00
5/8	4.01

Tabela 3: Não há tendência suave — proporções bem alinhadas em binário (1/4, 1/2) retornam à constante baixa ≈4; 1/3 é o pior caso observado.

Causa (hipótese original, ver correção abaixo na Seção 9.1): o teste de homogeneidade por amostragem (cantos + centro) funciona perfeitamente quando as bordas do bloco caem sobre fronteiras de subdivisão (potências de 2), mas força subdivisões extras quando k e $c = (L - k)/2$ não são bem alinhados. $k/L = 1/3$ é mau alinhamento; $k/L \in \{1/4, 1/2\}$ é bem alinhado.

Consequência: não existe fórmula fechada simples nós$(L, k) = f(k/L) \cdot L$ válida para todo k. Uma afirmação anterior (nós$(L, k) = 3L - k$, de um experimento diferente com teste exato) foi removida por não corresponder a esta implementação. O crescimento O(L) permanece válido; a constante de proporcionalidade depende do alinhamento binário de k (ver Seção 9.1 — não da heurística de amostragem, como se pensava originalmente).

1.9.1 9.1 Correção: a causa não é a heurística de amostragem

Esta subseção corrige a Seção 9 original. A hipótese de que o viés vinha de falsos positivos da amostragem por cantos+centro foi testada diretamente: implementou-se um teste de homogeneidade **geométrico exato** (interseção analítica reta-retângulo entre a região e as fronteiras conhecidas — hipotenusa e bordas do bloco; ver o script de validação e o módulo correspondente na engine), que por construção nunca produz falso positivo.

Resultado: o teste exato produz **exatamente os mesmos números de nós** que a amostragem heurística, para todos os casos testados — a Tabela 1 inteira (L=16 a 4096) e a Tabela 3 inteira (k/L de 1/6 a 5/8), com diferença zero em todos os casos de contagem de nós, e diferença $\leq 0{,}005$ em nós/L nos casos fracionários (arredondamento).

k/L	heurística (nós/L)	exato (nós/L)	diferença
1/4	4,000	4,001	0,001
1/6	5,320	5,323	0,003
1/3	6,660	6,655	-0,005
3/8	4,010	4,006	-0,004
1/2	4,000	4,000	0,000
5/8	4,010	4,014	0,004

Conclusão corrigida: a amostragem por cantos+centro já era, na prática, exata para esta geometria específica (fronteiras retas simples: uma diagonal e um quadrado

axis-aligned). A sensibilidade de nós/L ao alinhamento de k/L **não é um artefato da heurística de amostragem** — é uma propriedade geométrica real de como a posição do bloco central interage com a grade de subdivisão binária do quadtree (potências de 2). Frações bem alinhadas a potências de 2 ($k/L \in \{1/4, 1/2\}$) sempre vão gerar menos nós que frações mal alinhadas ($k/L = 1/3$), **mesmo com um teste de homogeneidade perfeito**. Validado por reconstrução ponto a ponto (0 falhas em 3000 amostras aleatórias, múltiplos (L, k)) e por paridade C++/Python.

1.10 10. Hexágono: dois trapézios colados pela hipotenusa

Removendo o triângulo menor de lado k (a "ponta") do triângulo de lado L, sobra um trapézio T_1 (base curta $x + y = k$, base larga $x + y = L - 1$). Refletindo T_1 pela hipotenusa — $(x, y) \mapsto (C - y, C - x)$, $C = L - 1$ — obtém-se T_2, colado a T_1 pela base larga compartilhada: um hexágono simétrico.

$$|T_1| = \sum_{i=k+1}^{L} i = \frac{L(L+1) - k(k+1)}{2}$$

$$N_{hex}(L, k) = 2|T_1| - L = L^2 - k(k+1)$$

Validado contra construção explícita para 30 combinações (L, k), $L \in [5, 100]$ — todos exatos:

L	k	pontos (construção real)	L² − k(k+1)
9	3	69	69
20	9	310	310
50	24	1 900	1 900
100	49	7 550	7 550

Leitura geométrica direta: L^2 é a malha quadrada completa; $k(k+1) = k^2 + k$ são as duas cunhas triangulares retiradas dos cantos — o hexágono é "o quadrado $L \times L$ menos duas pontas".

1.11 11. Efeito telescópico entre valores de k (gnomons generalizados)

Fixando L e variando k:

$$N_{hex}(L, k) - N_{hex}(L, k+1) = 2(k+1)$$

Análogo direto dos números gnomon clássicos. A soma das cascas de $k = 0$ até $k = m - 1$ reconstrói o salto direto:

$$\sum_{k=0}^{m-1} 2(k+1) = m(m+1) = N_{hex}(L, 0) - N_{hex}(L, m)$$

Confirmado numericamente (L=100, m=10: soma das cascas = 110, salto direto = 110).

1.12 12. Comparação com dados de ocupação esparsa (esfera e terreno sintéticos)

1.12.1 12.1 Por que dados sintéticos

Sem acesso a datasets reais neste ambiente, usam-se dados sintéticos que reproduzem características estruturais: esfera num cubo L^3 (proxy de órgão/tumor, fronteira curva suave), terreno por heightmap suave (proxy de voxels de jogo), e ruído puro como controle.

1.12.2 12.2 Extensão do método para 3D

O quadtree é generalizado para octree: nó cúbico de lado s subdivide em 8 filhos de lado $s/2$ quando não homogêneo (mesmo critério de amostragem, exato para $s \leq 2$).

1.12.3 12.3 Resultados

L	nós (esfera)	nós/L²	nós (terreno)	nós/L²
16	905	3.54	1 225	4.79
32	3 641	3.56	4 713	4.60
64	14 377	3.51	22 489	5.49
128	57 769	3.53	93 585	5.71

Tabela 5: A razão nós/L² estabiliza para ambos, confirmando O(L²). **(Confirmado por execução direta de `octree3d.py` — bate exato em todas as linhas.)**

Construção	Dimensão	Crescimento	Razão nós/bruto
Malha algébrica (Seção 5)	2D	O(L)	0,052
Esfera sintética	3D	O(L²)	0,028
Terreno sintético	3D	O(L²)	0,045
Ruído puro (sem estrutura)	3D	O(L³)	1,123 (pior que bruto!)

Tabela 6: Comparação final. ***(Confirmado: 0.0275≈0.028; 0.0446≈0.045; 1.1234≈1.123; malha algébrica 0.0501≈0.052 para L=128.)***

1.12.4 12.4 Interpretação

O ganho em dados sintéticos "realistas" é da mesma ordem de grandeza do obtido na construção algébrica. O princípio (compressão por homogeneidade local, custo proporcional à fronteira) generaliza para qualquer dado com fronteiras bem comportadas — não é peculiaridade do semiprimo.

O ruído puro é o contraponto: sem estrutura a explorar, o octree piora em relação ao array bruto (razão > 1), porque cada subdivisão adiciona um nó interno sem atingir folha homogênea maior que $1 \times 1 \times 1$.

1.12.5 12.5 O piso teórico para dados incompressíveis: um terceiro tipo de truncagem

A razão > 1 para ruído puro não é artefato — é limite matemático exato. Para dados verdadeiramente incompressíveis, a árvore desce até folhas $1 \times 1 \times 1$ em todo lugar:

$$N(d) = \sum_{i=0}^{d} b^i = \frac{b^{d+1} - 1}{b - 1}, \quad b^d = \text{raw}$$

$$\frac{N(d)}{b^d} = \frac{b}{b-1} - \underbrace{\frac{b^{-d}}{b-1}}_{\text{termo de truncagem}} \xrightarrow{d\to\infty} \frac{b}{b-1}$$

Para octree ($b = 8$): limite $8/7 \approx 1{,}142857$. Medido em $L = 8$ ($d = 3$) com verificação exata de homogeneidade: $1{,}142578$ — bate a fórmula exatamente.

Nota de validação (não presente no relatório original, verificada nesta consolidação): essa igualdade exata só se sustenta com dados verdadeiramente incompressíveis — em teste com valores contínuos (float aleatório, sem chance de coincidência), a medição bate $1{,}142578$ exatamente, confirmado também para $d = 1..5$ contra a fórmula fechada e contra o termo residual $b^{-d}/(b-1)$, todos exatos. **Com dados binários (0/1, que é o que `make_noise_occ` de fato gera)**, a razão medida fica sistematicamente **abaixo** do piso teórico (testado: $1{,}126953$ em vez de $1{,}142578$ para os mesmos parâmetros), porque blocos de 8 células têm ~0,8% de chance de sair homogêneo por coincidência mesmo sendo ruído puro. Isso não invalida a fórmula — ela descreve o pior caso absoluto — mas o valor específico citado como "medido" no relatório pressupõe um alfabeto de valores efetivamente contínuo, não o gerador binário incluído em `octree3d.py`.

Essa truncagem é qualitativamente diferente das da Seção 7: não é escolha de projeto, é consequência inevitável de qualquer representação finita — um viés sistemático, quantificável e residual, presente em qualquer estrutura em árvore aplicada a dados finitos.

1.13 13. Custo amortizado: construção única vs. renderização por quadro

1.13.1 13.1 Por que contagem de nós não é a métrica certa

O caso de uso real (renderização, fatiamento de volumes) não é sobre armazenar — é sobre visitar/pintar o volume repetidamente, quadro a quadro.

1.13.2 13.2 Metodologia

Compara-se **array denso** (percorre todas as L^3 células do zero a cada quadro) contra **octree** (construído uma vez; cada quadro seguinte apenas percorre a árvore pronta).

Cenário	build (1×)	denso/quadro	árvore/quadro	ganho/quadro
Esfera L=32	20,7 ms	9,3 ms	0,32 ms	29×
Terreno L=32	76,1 ms	46,0 ms	0,43 ms	107×

Cenário	build (1×)	denso/quadro	árvore/quadro	ganho/quadro
Esfera L=64	78,0 ms	77,8 ms	1,54 ms	51×
Terreno L=64	383,9 ms	357,9 ms	2,39 ms	150×

Tabela 7: Python puro, 20 quadros por cenário. **(Confirmado por execução de `render_bench2.py` — mesma ordem de grandeza e conclusão qualitativa; tempos absolutos variam por máquina, como esperado.)**

1.13.3 13.3 Ponto de equilíbrio (break-even)

No cenário mais custoso (terreno, L=64): build (384 ms) equivale a pouco mais de 1 quadro de array denso (358 ms/quadro) — a partir do segundo quadro a árvore já está adiante. Em todos os cenários, custo total (build + 20 quadros) ficou entre 4 e 16× menor que o array denso.

1.13.4 13.4 Interpretação

O ganho não vem de acesso mais rápido a um ponto isolado (é O(log L) na árvore contra O(1) no array denso — mais lento nessa operação específica). Vem do reuso amortizado de um trabalho de homogeneização pago uma única vez, ao longo de muitos quadros sobre um dado que não muda a cada instante. É por isso que motores gráficos constroem a estrutura esparsa ao carregar a cena e reaproveitam em todos os quadros seguintes.

1.14 14. O triângulo como figura de referência (Altura Máxima e Projeção Quadrada)

(Seções 14–16 vêm do adendo original — lá numeradas 13–15 — renumeradas aqui para encaixar após a expansão do relatório principal.)

Entre as figuras derivadas da malha (triângulo, trapézio, hexágono, ampulheta), o triângulo original — antes de qualquer corte ou dobra — é o de maior altura. Todas as demais figuras (Seção 10) são obtidas subtraindo ou recombinando partes do triângulo original; como nenhuma operação acrescenta pontos novos fora da malha de partida, nenhuma pode superar a altura do triângulo original — ele é o teto geométrico do qual as outras derivam.

1.14.1 14.1 A projeção como quadrado

O triângulo retângulo de catetos L tem como bounding box exatamente o quadrado $L \times L$: a projeção sobre cada eixo cobre $[0, L)$, então a menor região retangular capaz de conter o triângulo é o quadrado de área L^2. O próprio triângulo ocupa metade dessa projeção:

$$|T_{\triangle}| = \frac{L(L+1)}{2} \approx \frac{L^2}{2}$$

com a diferença de $L/2$ vindo dos pontos da própria hipotenusa discreta, contados uma única vez.

1.14.2 14.2 O quadrado como referência de medida

Essa projeção fornece uma unidade de medida comum: em vez de medir cada figura isoladamente, mede-se sua diferença em relação ao quadrado L^2 da figura de maior altura. Exatamente a leitura já obtida algebricamente na Seção 10 para o hexágono:

$$N_{hex}(L,k) = L^2 - k(k+1)$$

— o quadrado funcionando como referência absoluta, com as figuras derivadas descritas por subtração, não recontadas ponto a ponto.

1.14.3 14.3 Consequência prática

Ao tomar a figura de maior altura como referência de projeção, medir um conjunto de figuras relacionadas se reduz a uma quantidade fixa (L^2) mais termos de correção específicos de cada corte ou dobra — mesmo princípio de eficiência da Seção 4.

1.14.4 14.4 Nota conceitual: medir com referência não muda a figura

Usar L^2 como referência não altera a figura medida — o hexágono continua tendo exatamente $L^2 - k(k+1)$ pontos, quer descrito por subtração ou contagem direta. O que muda é apenas o custo de obter a contagem: com a referência, uma subtração; sem ela, enumeração região por região. A figura é a mesma nos dois casos; o ganho é inteiramente no processo de cálculo, não no objeto calculado.

1.15 15. Relações Métricas: Leitura Associativa vs. Fórmula

1.15.1 15.1 O teorema

Num triângulo retângulo, a altura relativa à hipotenusa o divide em dois triângulos menores, semelhantes entre si e ao original. Sejam b, c os catetos, a a hipotenusa, h a altura relativa a a, m, n as projeções de c e b sobre a ($m + n = a$):

$$b^2 = a \cdot n, \quad c^2 = a \cdot m, \quad h^2 = m \cdot n, \quad a \cdot h = b \cdot c$$

Somando as duas primeiras: $b^2 + c^2 = a(m+n) = a^2$ — o Teorema de Pitágoras emerge da semelhança, não como ponto de partida independente.

1.15.2 15.2 O que essas relações realmente são: associação antes de fórmula

$b^2 = a \cdot n$ não é, na origem, uma sentença algébrica a memorizar — é a tradução de uma percepção visual: o quadrado de lado b tem a mesma área que o retângulo de lados a e n, percebida pelo encaixe dos triângulos semelhantes. A fórmula é o registro posterior de uma associação já vista.

Mesmo tipo de leitura que perpassa todo o relatório: perceber que o hexágono é o quadrado menos duas pontas (Seção 10); perceber que o triângulo, por ter maior

altura, projeta naturalmente o quadrado L^2 (Seção 14) — a mesma leitura associativa aplicada à figura de referência do relatório inteiro.

1.15.3 15.3 Por que essa distinção importa

Reconhecer que a fórmula é tradução, não fonte, muda a ordem natural do trabalho: buscar primeiro a associação visual (semelhança, encaixe, projeção), e só então traduzir em símbolos para comunicar ou verificar.

1.16 16. Recuperação Geométrico-Métrica: Método da Partição e Escala

1.16.1 16.1 Premissas

Dadas a área A de um retângulo (ou $A_T = A/2$ do triângulo correspondente) e sua diagonal D, existe único par $a \geq b > 0$ com $a \cdot b = A$, $a^2 + b^2 = D^2$. Objetivo: recuperar a e b sem a equação quadrática explícita $x^2 - Sx + A = 0$.

1.16.2 16.2 Princípio da escala base

Usa-se um triângulo de referência (u, v, w) com propriedades conhecidas. Se a figura real se relaciona com a referência por fator de escala k, a área escala quadraticamente: Área real $= k^2 \cdot A_0$. Escolhendo $w = D$, tem-se $k = 1$, eliminando a necessidade de extrair raiz para obter o fator de escala.

1.16.3 16.3 Recuperação por partição e truncamento

Altura relativa à diagonal: $h_D = \dfrac{2A_T}{D} = \dfrac{A}{D}$ — medida linear por divisão simples, sem raízes.

Invariante do quadrado da soma: $(a + b)^2 = D^2 + 2A$ — recupera $(a + b)$ sem calcular a, b separadamente. Perímetro: $P = 2(a + b) = 2\sqrt{D^2 + 2A}$.

1.16.4 16.4 Síntese do algoritmo

Passo	Operação geométrica	Expressão	Exemplo (A=60, D=13)
1	Altura relativa à diagonal	$h_D = A/D$	$60/13$
2	Quadrado composto	$Q = D^2 + 2A$	$169 + 120 = 289$
3	Semiperímetro	$S = \sqrt{Q}$	17
4	Perímetro total	$P = 2S$	34

Validado em C++ e Python nesta consolidação: bate exato o exemplo do documento ($a = 12$, $b = 5$, $P = 34$) e passa em 2000/2000 casos aleatórios (a única "falha" observada em teste anterior foi ruído de ponto flutuante de $\sim 10^{-6}$ no caso degenerado $a = b$, não erro do método). Já integrado ao módulo `Render::BoundingReference` da engine.

1.16.5 16.5 Vantagem e generalização

O método evita montar a equação de segundo grau, substituindo-a por análise de proporção, restrições lineares e decomposição por partições — mesma linha da Seção 15: a raiz quadrada, quando aparece (Passo 3), surge de reconhecer um quadrado perfeito por partição ($289 = 17^2$), não de extrair raiz arbitrária. Generaliza para qualquer prisma ou volume cuja base seja parametrizada por projeções e razões fixas de área — a mesma lógica de escala (k^2 para área, k^3 para volume) que rege semelhança de figuras em qualquer dimensão.

1.17 17. Próximos passos

A implementação atual usa teste de homogeneidade por amostragem (cantos + centro), suficiente para fronteiras retas simples mas não generalizável a fronteiras curvas ou regiões sobrepostas. Direções de continuação, com status atualizado nesta consolidação:

- **Testes de homogeneidade exatos** via geometria analítica (interseção reta-retângulo) — [OK] **implementado e validado** (`py/homogeneidade_exata.py`, `render::SpatialPart` na engine). Resultado inesperado: não elimina o viés de alinhamento binário descrito na Seção 9 — confirma que esse viés é geométrico real, não artefato da amostragem (ver Seção 9.1, correção).
- **Extensão a octree 3D completo** — [OK] **concluído** (Seção 12, `octree3d.py`), embora ainda usando o mesmo teste de homogeneidade por amostragem (não geométrico exato) da Seção 5.
- **Esquema híbrido** (grafo de fusão da Seção 8 para classificação grosseira O(1), refinado por quadtree da Seção 5) — [OK] **implementado** (`src/stream/hybrid_partition.hpp` da engine, ver Seção 18) e **testado com resultado negativo**: para a malha do relatório (L=256, k=85), o híbrido usa **2010 nós contra 1673 do quadtree puro** — pior, não melhor. Causa: a fusão grosseira separa "bloco central" (bounding box pequena) de "resto da malha" (T1+T2 nos dois cantos opostos), mas a bounding box desse segundo componente degenera para quase o tamanho da malha inteira, anulando o ganho do Passo 1. O híbrido só compensa quando os componentes fundidos são geometricamente compactos — não é o caso desta construção específica.
- **Dados de ocupação esparsa realistas** (voxels de jogos, volumes médicos) — parcialmente endereçado: dados sintéticos (esfera, terreno) implementados e validados (Seção 12); dados reais permanecem como continuação, por falta de acesso à rede neste ambiente.
- **Validação do piso teórico para dados incompressíveis (Seção 12.5)** — a fórmula está confirmada matematicamente, mas o script de geração de ruído (`make_noise_occ`) usa alfabeto binário, que produz medições sistematicamente abaixo do piso teórico por coincidências de homogeneidade local; um gerador de valores contínuos seria necessário para reproduzir o valor específico citado como "medido" no relatório.

1.18 18. Status da implementação em engine (C++)

Paralelamente à validação em Python (Seções 5–13), a mesma matemática foi portada para uma engine em C++20 (`engine-fluxo/src/`), estruturada como um Render Loop central que consome os módulos abaixo em ordem fixa a cada quadro. O objetivo desta seção é manter rastreável a relação entre teoria (este relatório) e implementação — cada módulo abaixo cita a seção de origem, e o `README.md` do repositório mantém o log cronológico de mudanças.

Módulo C++	Seção de origem	Status
`core::FastMath` (`TruncaPorMascara`, `TruncaPorShift`)	Seção 7	[OK] implementado, testado
`stream::UnionFind`	Seção 8	[OK] implementado, réplica de `merge_grafo.py`
`stream::SpatialPartition` (Quad/Octree, heurística)	Seções 5, 12	[OK] implementado
`stream::SpatialPartitionExact` (geométrico exato)	Seções 9, 17	[OK] implementado, paridade total com a heurística (ver Seção 9.1)
`stream::HybridPartition`	Seção 17	[OK] implementado, resultado negativo confirmado (ver acima)
`render::MetricRecovery`	Seção 16	[OK] implementado, 0 falhas em 2000 casos aleatórios (C++)
`render::BoundingReference` (L^2 − correção)	Seção 13	[OK] implementado, usa `MetricRecovery`
`stream::WindowController` (ring buffer)	— (arquitetura)	[OK] implementado
`core::HardwareProfiler`	— (arquitetura)	[stub] stub, valores fixos (VRAM/PCIe hardcoded)
`render::ProjectivePipeline`	— (arquitetura)	[stub] stub, sem API gráfica real (Vulkan/DX12)
`stream::SliceStreamer`	Seções 5, 9	[stub] classifica via `SpatialPartitionExact` (real), mas malha ainda hardcoded (não lê disco)
Teste de homogeneidade geométrico exato	Seção 9	[OK] implementado — ver correção da Seção 9.1

Todos os módulos marcados [OK] compilam com `g++ -std=c++20` e têm resultado numérico conferido contra os scripts Python correspondentes. O relatório completo de mudanças, incluindo o que foi recuperado após perda parcial de contexto entre sessões, está em `engine-fluxo/README.md`.